Федеральное государственное автономное образовательное учреждение высшего образования «Научно-образовательная корпорация ИТМО»

Факультет программной инженерии и компьютерной техники Направление подготовки 09.03.04 Программная инженерия

Отчёт по лабораторной работе №2

По дисциплине «Веб-программирование» (3 семестр)

Студент:

Розова Елена

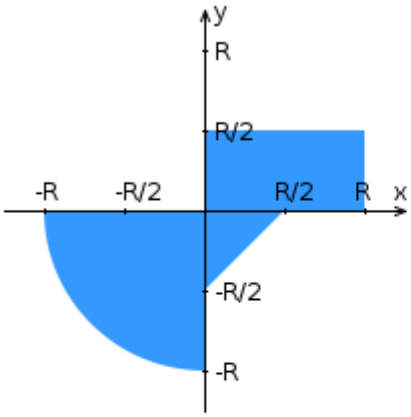
Группа Практик:

Дениченко Александр Олегович

1. Введение 1
1.1 Задание на лабораторную 2
1.2 Основные технологии 3
1.3 Особенности технологий 3
1.4 Описание дополнительного задания 3
2 Аналитика и проектирование 4
2.1 Описание интерфейсов 4
2.1.1 Диаграмма C4 level 1 4
2.1.2 Диаграмма DFD 4
3 Выполнение 5
3.1 Реализация интерфейсов 5
3.2 Реализация остального решения 5
3.3 Примеры запросов и ответов 6
3.4 Реализация дополнительного задания 6
4 Выполнение 6
4.1 Выводы по лабораторной работе 6
4.1.1 Зачем 6
4.1.2 Как 6
4.1.3 Чему научились 6
4.2 Выводы по дополнительному заданию 6
4.2.1 Зачем 6
4.2.2 Как 7
4.2.3 Чему научились 7
5 Источники информации 7
5.1 ИИ 7
5.2 Другие источники 7

1. Введение

1.1 Задание на лабораторную



изменение X: Button {'-3','-2','-1','0','1','2','3','4','5'}

изменение Y: Text (-3 ... 3)

Изменение R: Button {'1','2','3','4', '5'}

Доработать ЛР №1 и разработать FastCGI сервер на языке Java, определяющий попадание точки на координатной плоскости в заданную область, и создать HTML-страницу, которая формирует данные для отправки их на обработку этому серверу.

Параметр R и координаты точки должны передаваться серверу посредством HTTP-запроса. Сервер должен выполнять валидацию данных и возвращать HTML-страницу с таблицей, содержащей полученные параметры и результат вычислений — факт попадания или непопадания точки в область (допускается в ответе сервера возвращать json строку, вместо html-страницы). Предыдущие результаты должны сохраняться между запросами и отображаться в таблице.

Кроме того, ответ должен содержать данные о текущем времени и времени работы скрипта.

Комментарии по выполнению ЛР:

- Требуется поднять Apache httpd веб-сервер от лица своего пользователя на гелиосе (шаблон файла конфигурации доступен для скачивания наверху страницы)
- Веб-сервер должен заниматься обслуживанием статического контента (html, css, js) и перенаправлять запросы за динамическим контентом к FastCGI серверу

- FastCGI сервер требуется реализовать на языке Java (полезная библиотека в помощь в виде jar архива доступна для скачивания наверху страницы) и поднять также на гелиосе
- **Путем обращений из JavaScript к FastCGI серверу требуется показать понимание принципа AJAX**

Разработанная HTML-страница должна удовлетворять следующим требованиям (дополнительными, по отношению к ЛР №1, различаются по варианту):

- Данные формы должны передаваться на обработку посредством GET-запроса.

1.2 Основные технологии

Java SE (FastCGI SDK): Применяется на стороне бэкенда для развертывания легковесного демона, обрабатывающего бинарные пакеты протокола FastCGI через локальный TCP-порт.

Apache httpd (модуль proxy_fcgi): Выступает в роли фронтенд-веб-сервера для маршрутизации внешнего веб-трафика и терминирования статических файлов.

JavaScript (ECMAScript 6+ / AJAX API): Используется на стороне клиента для асинхронного взаимодействия с сервером без перезагрузки интерфейса.

HTML5 & CSS3: Применяются для декларативного описания структуры интерфейса с использованием табличной разметки и адаптивной стилизации элементов управления.

1.3 Особенности технологий

Протокол FastCGI: В отличие от классического CGI, который создает отдельный процесс операционной системы на каждый HTTP-запрос (что порождает высокие накладные расходы ресурсов), FastCGI использует постоянно запущенный фоновый процесс (демон). Связь между веб-сервером и FastCGI-приложением поддерживается через постоянные TCP-соединения или Unix-сокеты, что кардинально снижает задержки и повышает производительность системы.

Асинхронные GET-запросы (AJAX / Fetch API): GET-запросы передают параметры состояния системы (\(X, Y, R\)) непосредственно внутри строки URL (Query String). Применение AJAX позволяет браузеру выполнять фоновый обмен данными с сервером, сохраняя текущее состояние DOM-дерева и обеспечивая мгновенный интерфейсный отклик для пользователя.

1.4 Описание дополнительного задания

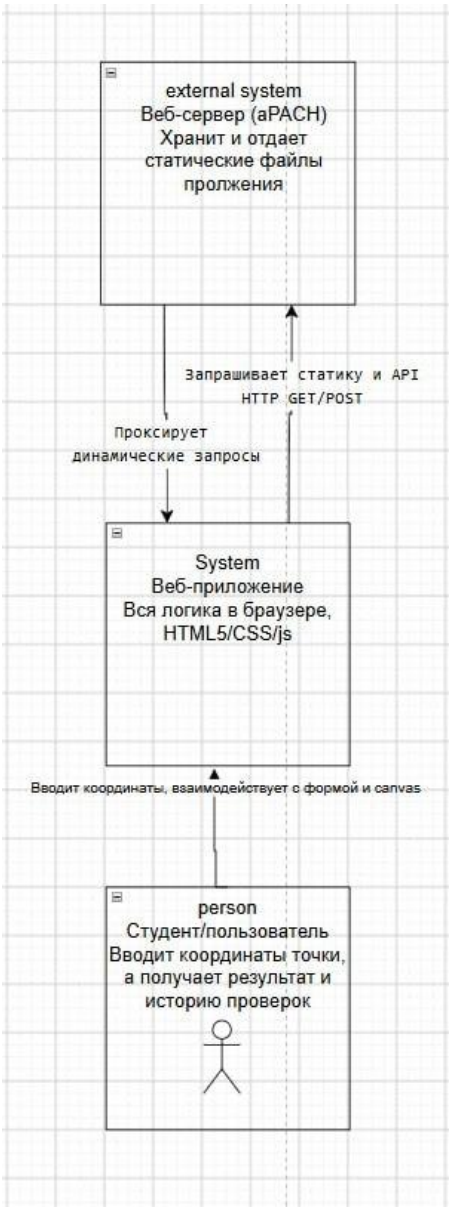
В 2 лабе требуется график сделать в 3d, чтобы можно было крутить вертеть

Установка точек должна имитировать взрывы

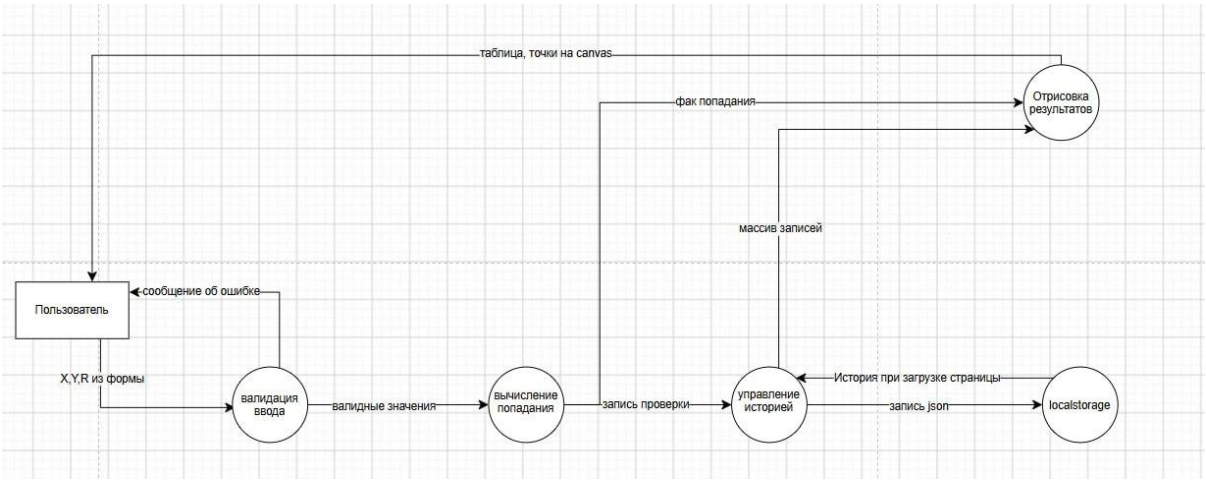
2 Аналитика и проектирование

2.1 Описание интерфейсов

2.1.1 Диаграмма C4 level 1



2.1.2 Диаграмма DFD



3 Выполнение

3.1 Реализация интерфейсов

Интерфейс пользователя спроектирован с жестким разделением логических слоев на базе семантической верстки. Основные структурные блоки:

- **Шапка документа (Header):** Содержит ФИО студента, номер группы и номер варианта, жестко зафиксированные в верхней части страницы.
- **Интерактивная форма ввода:** Содержит группу переключателей/полей для координат \(X\), текстовое поле для координаты \(Y\) (с ограничением длины и регулярными выражениями для отсечения букв), а также элементы выбора радиуса \(R\).
- **Динамическая таблица результатов:** Отображает всю историю запросов в текущей сессии (Параметры \(X, Y, R\), результат проверки, текущее время, время обработки).

3.2 Реализация остального решения

- **Логика разбора GET-запроса на бэкенде:** Так как данные передаются методом GET, Java-сервер считывает переменную окружения `QUERY_STRING`. Строка разбивается по разделителю &, после чего ключи и значения декодируются из URL-формата.
- **Алгоритм проверки области:** Координаты проверяются на вхождение в систему геометрических неравенств, описывающих заштрихованные фигуры на графике (сектор круга, прямоугольник, прямоугольный треугольник).
- **Жизненный цикл FastCGI процесса:** Сервер запускает бесконечный цикл обработки `FCGIInterface.FCGIaccept()`. При каждом итерационном шаге приложение считывает входные параметры, выполняет расчеты, отправляет заголовки `Content-Type: application/json` и тело ответа в `System.out`, после чего закрывает текущий поток вывода для завершения HTTP-транзакции.

3.3 Примеры запросов и ответов

Запрос: GET /fcgi-bin/server.jar?x=0&y=0.5&r=2 HTTP/1.1

Ответ:


```
{
  "status": "success",
  "x": 0.0,
  "y": 0.5,
  "r": 2.0,
```

"result": "ПОПАДАНИЕ",

"currentTime": "11:02:15",

"scriptTime": "0.42ms"

}

3.4 Реализация дополнительного задания

См в примере выполнения

4 Выполнение

https://github.com/elenarozova/ITMO/tree/main/Second/web/lab2

4.1 Выводы по лабораторной работе

4.1.1 Зачем

Изучение FastCGI, CGI, поиск разницы наличие веб-сервера и его отсутствие, понимание асинхроннвх запросов

4.1.2 Как

Первыя версия без допа была запущена и на гелиусе и в докер-контейнере, последующая с допом только в докер контейнере

4.1.3 Чему научились

Запускать докер контейнер, делать докер compose

4.2 Выводы по дополнительному заданию

4.2.1 Зачем

Прикольно

4.2.2 Как

С помощью библиоткеи tree

4.2.3 Чему научились

Делать 3d графики

5 Источники информации

5.1 ИИ

ИИ был использован для:

1. Поиска необходимой информации
2. Форматировании кода
3. Написание отчёта (частично)
4. Помощи нахождении ошибок в коде (логических, фактических и опечаток)
5. Выбора более красивого оформления в css

5.2 Другие источники

https://fastcgi-archives.github.io/fcgi2/doc/fcgi-java.htm

https://developer.mozilla.org/ru/